

Custom VPC Setup with Bastion Host and NAT Gateway

Project: Custom VPC Setup with Bastion Host and NAT Gateway

Services: VPC, EC2, Internet Gateway, NAT Gateway, Security Groups

Description: In this project, I created a custom VPC with both public and private subnets. I launched a bastion host in the public subnet to securely access a private EC2 instance. I also configured a NAT Gateway to allow internet access for the private instance without exposing it to the public internet.

Skills: VPC configuration, subnetting, routing tables, security groups, SSH access, secure architecture

Definition: This project is used to create a secure network architecture in AWS using a custom VPC. It includes public and private subnets, where the public subnet contains a bastion host for secure access, and the private subnet contains an EC2 instance without public access. A NAT Gateway is used to provide internet access to the private instance for updates and downloads while keeping it secure from direct external access.

Scope of the Project:

The scope of this project is to design and implement a secure AWS network architecture using VPC components. It focuses on controlling access between public and private resources.

This project covers:

- Creating a custom VPC with public and private subnets
- Launching a Bastion Host and a private EC2 instance for secure access
- Configuring Internet Gateway and NAT Gateway for connectivity
- Setting up security groups and routing for proper access control

Methodology:

1. Bastion Host + NAT Gateway (Main Method Used)

- Used VPC, EC2, Internet Gateway, and NAT Gateway
- Bastion host is used as a secure entry point to access private instance
- NAT Gateway allows private EC2 to access internet without public IP
- Provides secure and real-world architecture
- Easy to manage and commonly used in production
- NAT Gateway was created for testing and later deleted to avoid extra cost

2. Bastion Host + NAT Instance (Alternative Method)

- Uses EC2 instance instead of NAT Gateway
- Cost-effective and free-tier friendly
- Requires manual configuration (IP forwarding, iptables)
- Suitable for learning and small projects
- Slightly complex compared to NAT Gateway

AWS Services used in the Project:

1. Amazon VPC (Virtual Private Cloud)

- **Definition:** Amazon VPC is a service that allows users to create a private network in AWS.
- **Purpose:** To isolate resources in a secure network
- **Role:** Used to create public and private subnets for this project

2. Amazon EC2 (Elastic Compute Cloud)

- **Definition:** EC2 provides virtual servers in the cloud
- **Purpose:** To run applications and servers
- **Role:** Used to launch Bastion Host and Private EC2 instance

3. Internet Gateway

- **Definition:** Internet Gateway allows communication between VPC and internet
- **Purpose:** Provides internet access to public subnet
- **Role:** Attached to VPC to enable Bastion Host access

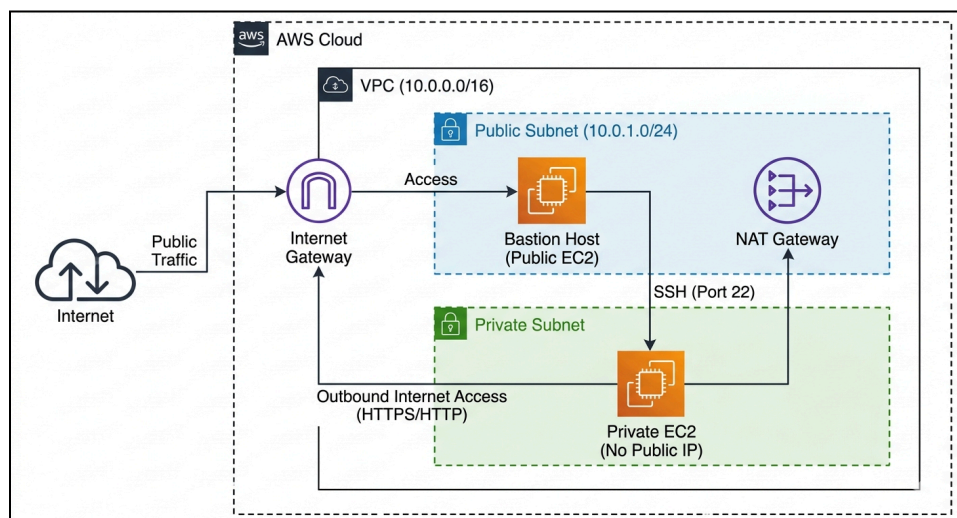
4. NAT Gateway

- **Definition:** NAT Gateway allows private instances to connect to internet without public IP
- **Purpose:** Secure outbound internet access
- **Role:** Used by private subnet to access internet (for updates, downloads)

5. Security Groups

- **Definition:** Security groups act as virtual firewalls for EC2 instances
- **Purpose:** Control inbound and outbound traffic
- **Role:** Bastion SG allows SSH from my IP ,Private EC2 allows SSH only from Bastion

Architecture Diagram:



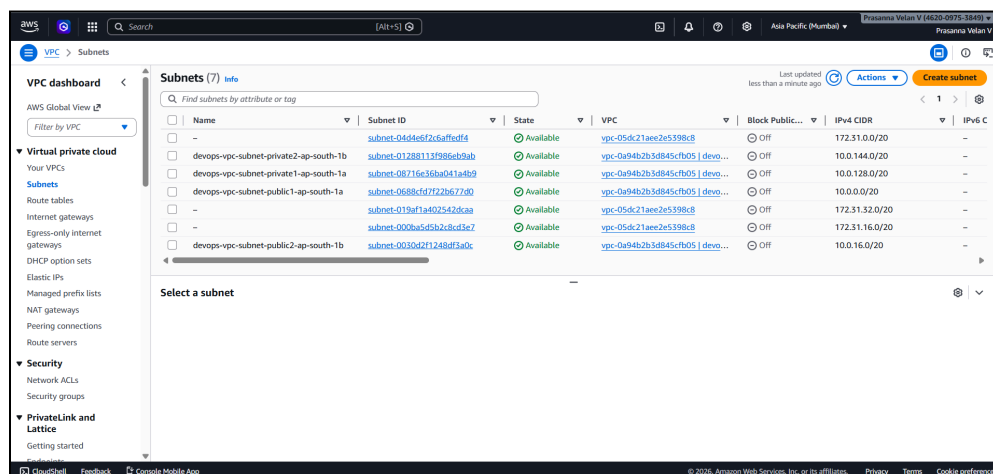
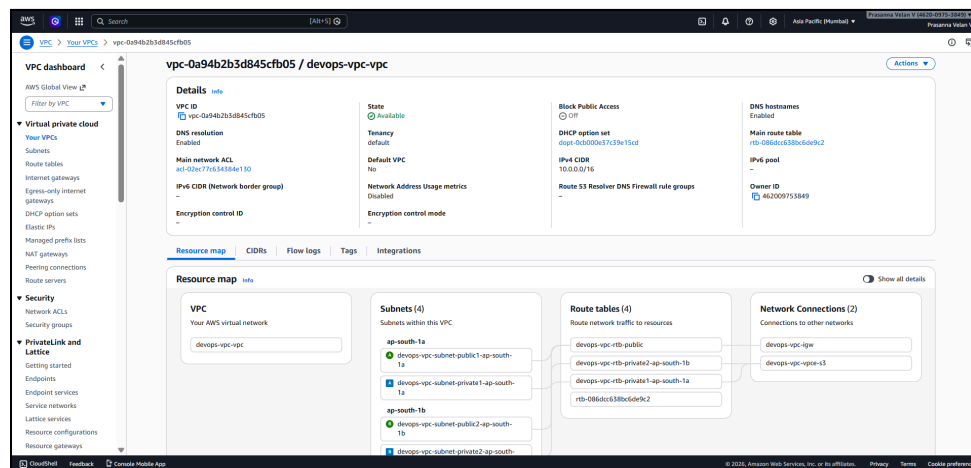
Implementation Steps to Do the Project:

Step 1: Create VPC using “VPC and More” Option

- Go to AWS Console → VPC Dashboard
- Click **Create VPC** then Select **VPC and more** option
- Enter Name: **devops-vpc**
- CIDR block: **10.0.0.0/16**
- Number of Availability Zones: 1 ,Number of public subnets: 1 and Number of private subnets: 1
- Enable Internet Gateway also Enable NAT Gateway (selected for testing purpose)
- Click **Create VPC**

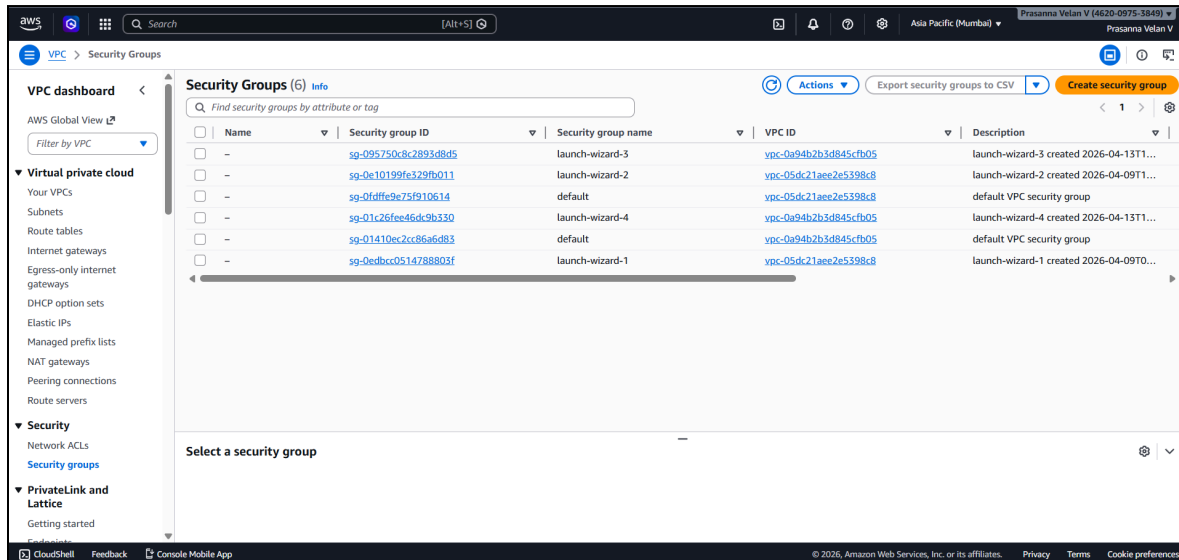
This automatically created:

- Public subnet
- Private subnet
- Route tables
- Internet Gateway
- NAT Gateway



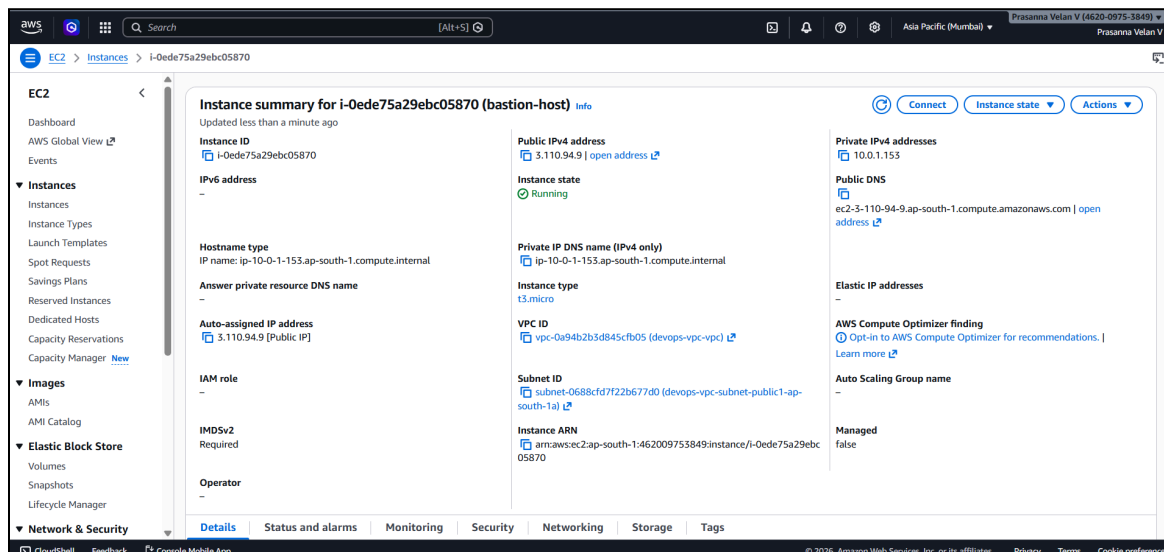
Step 2: Create Security Groups

- Created Bastion Security Group - Allowed SSH (port 22) from my IP
- Created Private EC2 Security Group - Allowed SSH only from Bastion Security Group



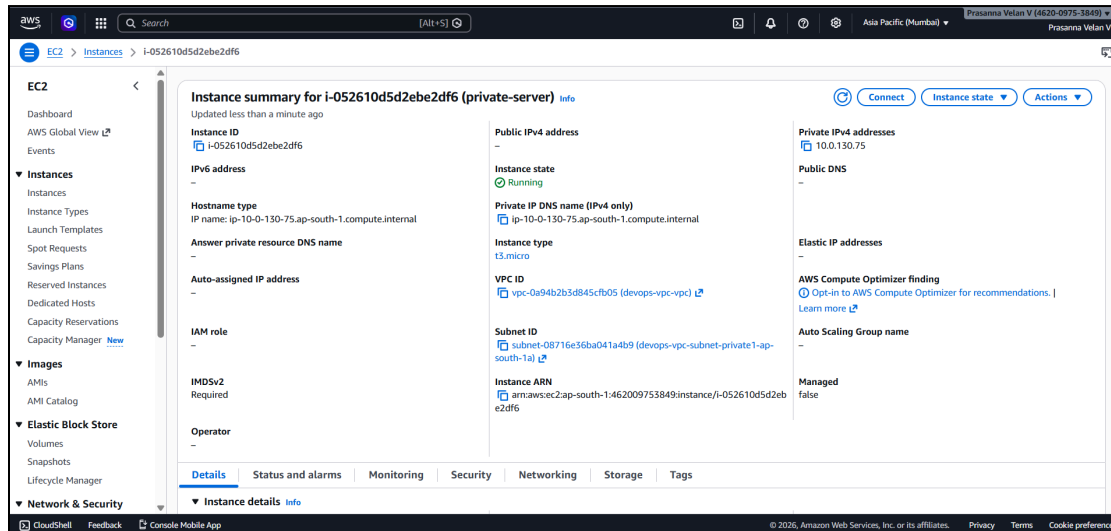
Step 3: Launch Bastion Host (Public EC2)

- Go to EC2 then Launch Instance
- Name: bastion-host and Select public subnet
- Enabled auto-assign public IP and Attach Bastion Security Group



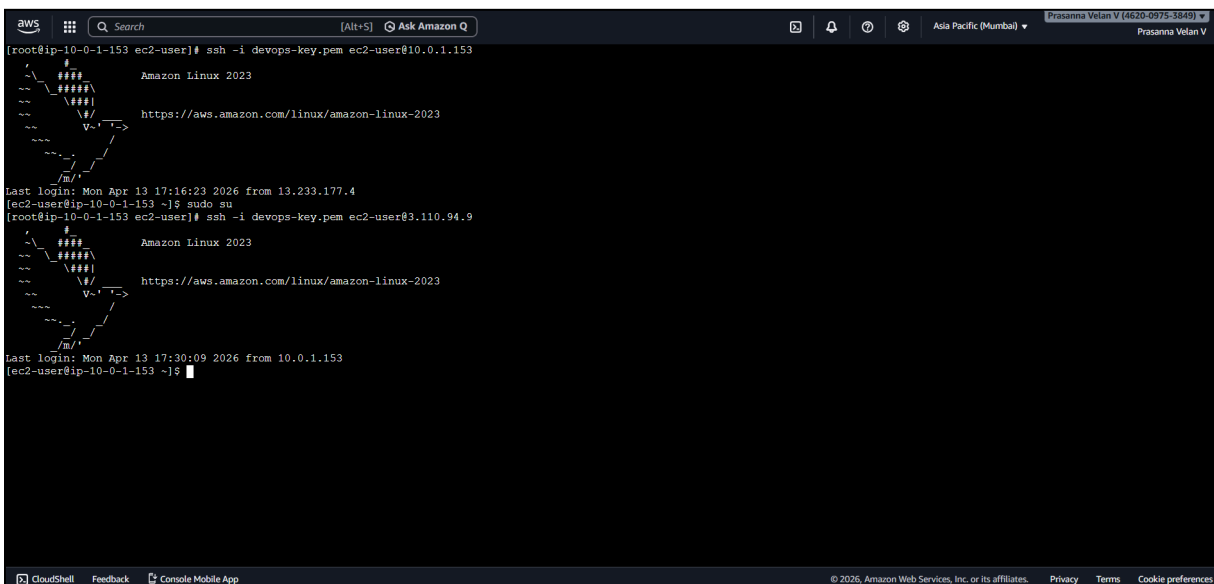
Step 4: Launch Private EC2 Instance

- Name: private-server and Select private subnet
- Disabled public IP and Attach Private Security Group



Step 5: SSH Access Setup

- Connected from local machine to Bastion Host using .pem file then Uploaded .pem file to Bastion
- From Bastion, connected to Private EC2 using private IP



Step 6: Test Internet Access

- Logged into private EC2
- Ran command: ping [google.com](https://www.google.com)

- Verified internet is working through NAT Gateway

```
[root@ip-10-0-130-75 ec2-user]# ping google.com
PING google.com (142.250.206.174) 56(84) bytes of data:
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=1 ttl=116 time=2.29 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=2 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=3 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=4 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=5 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=6 ttl=116 time=2.05 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=7 ttl=116 time=2.05 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=8 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=9 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=10 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=11 ttl=116 time=2.06 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=12 ttl=116 time=2.05 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=13 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=14 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=15 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=16 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=17 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=18 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=19 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=20 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=21 ttl=116 time=2.01 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=22 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=23 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=24 ttl=116 time=2.10 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=25 ttl=116 time=2.05 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=26 ttl=116 time=2.02 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=27 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=28 ttl=116 time=2.08 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=29 ttl=116 time=2.05 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=30 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=31 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=32 ttl=116 time=2.07 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=33 ttl=116 time=2.05 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=34 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=35 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=36 ttl=116 time=2.03 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=37 ttl=116 time=2.04 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=38 ttl=116 time=2.06 ms
64 bytes from dell1s22-in-f14.1e100.net (142.250.206.174): icmp_seq=39 ttl=116 time=2.03 ms
```

Troubleshooting:

Issue 1: SSH Key Not Found

- Error: No such file or directory
- Solution: Changed directory to correct path and uploaded .pem file

Issue 2: Permission Denied (publickey)

- Cause: Key file not present in bastion
- Solution: Uploaded key file and set permission using chmod 400

Issue 3: Unable to Connect to Private EC2

- Cause: Tried accessing from local machine
- Solution: Connected through Bastion host

Issue 4: SCP to Private IP Failed

- Cause: Private IP not accessible from internet
- Solution: Used Bastion as intermediate server

Conclusion:

This project helped me understand how to design a secure network in AWS using VPC. I created public and private subnets and used a bastion host to securely access the private EC2 instance. I also configured a NAT Gateway to allow internet access for the private instance without exposing it publicly.

This architecture improves security by restricting direct access to private resources. It is widely used in real-world applications. The project also helped me understand routing, subnetting, and secure SSH access in AWS.